TSP AI Route Optimizer

1. Tổng quan dự án

Tên đề tài: Ứng dụng trí tuệ nhân tạo tối ưu hóa tuyến đường giao hàng bằng thuật toán di truyền.

Dự án xây dựng một ứng dụng Python có giao diện web Streamlit để biểu diễn bài toán người bán hàng du lịch (Traveling Salesman Problem, TSP), tìm tuyến khép kín bằng Genetic Algorithm (GA), và đánh giá kết quả bằng các phương pháp đối chứng. Ứng dụng phục vụ học tập, thực nghiệm và phân tích tối ưu tổ hợp trên mặt phẳng tọa độ. Sản phẩm bao gồm mã nguồn chia module, kiểm thử tự động, dữ liệu CSV, bộ benchmark tái lập, biểu đồ tương tác và tài liệu.

2. Phát biểu bài toán

Cho một kho hàng và một tập khách hàng có tọa độ hai chiều. Cần tìm thứ tự ghé thăm sao cho mỗi khách hàng được thăm đúng một lần, xe xuất phát tại kho, quay lại kho và tổng khoảng cách là nhỏ nhất. Bài toán có nhiều hoán vị khả dĩ; duyệt toàn bộ không phù hợp khi số khách hàng tăng.

Trong báo cáo, n là **số khách hàng**, không gồm kho. Vì vậy, bài toán 30 khách hàng có tổng cộng 31 địa điểm. Khoảng cách là Euclid theo đơn vị tọa độ. Không diễn giải kết quả thành số kilômét đường bộ hoặc thời gian giao hàng khi chưa có dữ liệu tương ứng.

3. Mục tiêu

Dự án có ba nhóm mục tiêu. Về thuật toán, tự triển khai GA trên hoán vị, duy trì tuyến hợp lệ, hỗ trợ seed và lưu quá trình tiến hóa. Về phần mềm, cung cấp nhập/xuất dữ liệu, kiểm tra tham số, trực quan hóa tuyến, so sánh và benchmark trong một ứng dụng có cấu trúc rõ ràng. Về nghiên cứu, sử dụng kết quả đo thật, lưu dữ liệu thô và thảo luận cả trường hợp GA không vượt baseline.

4. Phạm vi

Hệ thống giải TSP Euclid đối xứng với một kho và một xe. Các thuật toán gồm GA thuần, Random Search, Nearest Neighbor (NN), Held-Karp cho tối đa 12 khách hàng và NN + 2-opt. Phần lõi hỗ trợ dataset có 1-500 khách hàng; giao diện và benchmark có giới hạn vận hành thấp hơn để duy trì tính đáp ứng.

Các bài toán nhiều xe, tải trọng, thời hạn giao hàng, mạng đường bộ, giao thông và tối ưu đa mục tiêu nằm ngoài phạm vi. GA không được trình bày như bộ giải có chứng nhận tối ưu. 2-opt được đặt tên riêng để phân biệt với GA thuần.

5. Cơ sở lý thuyết

GA là phương pháp tìm kiếm dựa trên quần thể, kết hợp lựa chọn, lai ghép, đột biến và giữ cá thể tốt. Biểu diễn có vai trò quyết định: chromosome nhị phân hoặc tọa độ tùy ý không trực tiếp bảo đảm ràng buộc TSP, trong khi hoán vị khách hàng cho phép mô tả mỗi lời giải hợp lệ. Nền tảng phương pháp tiến hóa được trình bày trong công trình của Holland [1].

NN là heuristic tham lam khai thác hình học địa phương. Random Search là baseline ngẫu nhiên có cấu trúc đơn giản, phù hợp kiểm tra liệu các toán tử tiến hóa có đem lại lợi ích so với lấy mẫu độc lập trên cùng ngân sách. Held-Karp sử dụng quy hoạch động theo tập con để tìm tối ưu toàn cục [2]. 2-opt là tìm kiếm cục bộ thay hai cạnh bằng cách đảo một đoạn tuyến.

6. Mô hình toán học

Gọi kho là d, tập khách hàng là V = {1, …, n}, tọa độ địa điểm i là (x_i, y_i). Ký hiệu d trong mô hình không yêu cầu kho nằm ở hàng đầu của bảng dữ liệu.

Khoảng cách giữa hai địa điểm:

c(i,j) = sqrt((x_i - x_j)^2 + (y_i - y_j)^2).

Với hoán vị π của V, hàm mục tiêu là:

min L(pi) = c(d,pi_1) + sum(k=1..n-1)c(pi_k,pi_(k+1)) + c(pi_n,d).

Các ràng buộc được thực hiện thông qua biểu diễn: mỗi khách hàng xuất hiện đúng một lần; không có khách hàng ngoài dataset; kho chỉ ở hai đầu; tập tọa độ không đổi khi thuật toán chạy. Hai ID khác nhau tại cùng tọa độ vẫn là hai khách hàng cần thăm. Dữ liệu có khoảng cách 0 vẫn hợp lệ.

7. Thiết kế Genetic Algorithm

Chromosome chứa chỉ số khách hàng, loại kho khỏi các toán tử di truyền. Quần thể ban đầu được tạo từ các hoán vị ngẫu nhiên, không chèn trước nghiệm NN. Bộ sinh ngẫu nhiên cục bộ dùng seed đã ghi nhận, tránh phụ thuộc trạng thái ngẫu nhiên toàn cục.

Tournament selection lấy một nhóm cá thể không hoàn lại trong mỗi lượt, chọn cá thể có khoảng cách nhỏ nhất. Order Crossover (OX) sao chép một đoạn của bố mẹ thứ nhất, sau đó điền các gene còn thiếu theo thứ tự vòng quanh bố mẹ thứ hai. Inversion mutation đảo một đoạn liên tiếp; cả hai toán tử giữ nguyên tập khách hàng. Crossover rate là xác suất áp dụng cho mỗi cặp bố mẹ; mutation rate là xác suất áp dụng cho mỗi cá thể con.

Elitism sao chép E cá thể tốt nhất và tái sử dụng giá trị mục tiêu đã tính. Quần thể được điền đủ bằng các con mới. Khoảng cách được dùng nhất quán cho minimization; fitness hiển thị là 1 / (distance + 10^-12). Average fitness là trung bình các fitness cá thể, không phải nghịch đảo average distance.

Lịch sử bao gồm thế hệ 0 và mỗi thế hệ đã hoàn thành: best distance, average distance, best fitness, average fitness, tuyến tốt nhất đã gặp và số đánh giá tích lũy. Thuật toán kết thúc khi hết số thế hệ hoặc patience không cải thiện nếu tùy chọn dừng sớm được bật. Với population P, generations G, không dừng sớm và elite E, số đánh giá là P + G(P-E). Đánh giá lại một hoán vị trùng vẫn được tính nếu thật sự tính hàm mục tiêu.

8. Thuật toán đối chứng

Random Search: sinh đúng ngân sách B hoán vị ngẫu nhiên, tính tổng khoảng cách và giữ tuyến tốt nhất. Seed độc lập với GA. Trong benchmark, B bằng số đánh giá thực tế của GA tương ứng.

Nearest Neighbor: từ kho, chọn khách hàng chưa thăm có khoảng cách nhỏ nhất; phá hòa theo chỉ số hàng; quay lại kho sau khi thăm hết. Kết quả xác định cho cùng dataset có thứ tự hàng cố định.

Held-Karp: trạng thái C(S,j) là chi phí tối thiểu đi từ kho, thăm đúng tập S, kết thúc ở j. Khởi tạo bằng khoảng cách kho-khách hàng. Truy hồi xét mọi tiền nhiệm trong S\{j}, rồi cộng cạnh về kho khi kết thúc. Truy vết cho tuyến tối ưu. Thời gian O(n^2 * 2^n), bộ nhớ O(n * 2^n) nên giới hạn cứng 12 khách hàng. Chỉ số evaluations của phương pháp này đếm chuyển trạng thái DP, không tương đương một tuyến ứng viên của GA.

NN + 2-opt: lấy tuyến NN, duyệt các cặp cạnh để đảo đoạn nếu làm giảm tổng khoảng cách. Dùng chiến lược first improvement và lặp đến khi không còn cải thiện theo cách duyệt. Runtime bao gồm tạo tuyến

NN. Phương pháp giữ kho ở hai đầu và không làm xấu tuyến khởi tạo; không chứng minh tối ưu toàn cục.

9. Kiến trúc hệ thống

Kiến trúc tách thành sáu lớp: giao diện Streamlit; mô hình/validation/nhập dữ liệu; ma trận khoảng cách; các bộ giải độc lập; đánh giá thực nghiệm; trực quan hóa. `app.py` khởi chạy ứng dụng, còn thuật toán nằm trong `src/tsp/algorithms/` và không phụ thuộc Streamlit.

`Location`, `Dataset` và `GAConfig` là mô hình bất biến, có validation. `OptimizationResult` thống nhất cấu trúc trả về giữa các bộ giải, gồm algorithm, route, distance, runtime_seconds, evaluations, seed, parameters và history. Route xuất ra là tuyến đầy đủ dùng chỉ số hàng dataset, bao gồm kho ở cả đầu và cuối. ID và tên chỉ phục vụ nhận diện/hiển thị.

10. Luồng dữ liệu

Dữ liệu từ CSV, bảng nhập, bộ mẫu hoặc seed đi qua validation để tạo dataset bất biến. Từ dataset, bộ giải tính ma trận Euclid và chạy tìm kiếm. Kết quả được lưu vào trạng thái phiên, sau đó dùng để tạo KPI, bảng, biểu đồ và tệp tải xuống.

Fingerprint nhận diện dataset bao gồm thứ tự hàng và nội dung. Khi áp dụng dataset mới, kết quả cũ cần được vô hiệu hóa. Thay đổi trang, cách xem biểu đồ hoặc thế hệ đang hiển thị chỉ đọc kết quả đã lưu; không tự chạy lại thuật toán. JSON lưu đủ dữ liệu để kiểm tra và dựng lại lịch sử; CSV phục vụ trao đổi tọa độ và số liệu.

11. Thiết kế giao diện

Giao diện theo hướng dashboard phân tích, sử dụng nền navy, card bo góc, màu indigo cho thao tác chính, cyan cho nhấn mạnh và màu riêng cho kho. Phân cấp chữ, khoảng cách giữa các vùng và nhãn đơn vị giúp đọc kết quả mà không cần hiểu chi tiết GA.

Bảy trang thực hiện các nhiệm vụ khác nhau: Dashboard tổng quan; Delivery Simulation phát lại tuyến bằng xe SVG; Route Playground quản lý tọa độ; GA Lab cấu hình và phát lại tiến hóa; Algorithm Comparison so sánh tuyến và chỉ số; Experiments & Analytics chạy nhiều seed; About & Documentation giải thích lý thuyết và giới hạn. Biểu đồ Plotly hỗ trợ hover, zoom, reset và xuất ảnh. Mô phỏng xe dùng tọa độ Euclid, không biểu diễn chuyển động trên mạng đường bộ.

12. Triển khai và bảo đảm chất lượng

Python phụ trách mô hình, thuật toán, kiểm thử và xử lý tệp. NumPy tính ma trận và đánh giá quần thể; Pandas đọc/ghi CSV và tổng hợp; Streamlit điều khiển giao diện; Plotly hiển thị tương tác; Matplotlib xuất PNG benchmark; Pytest kiểm tra tự động.

Validation kiểm tra đúng một kho, ID duy nhất, tên không rỗng, tọa độ số hữu hạn, số khách hàng hợp lệ, cấu hình GA và giới hạn exact. CSV boolean được phân tích rõ `true/false` hoặc `1/0`; không dùng cách ép chuỗi `false` thành truthy. Lỗi được báo ở ranh giới nhập/chạy để người dùng sửa dữ liệu.

Bản cơ sở trước khi thêm mô phỏng đã chạy **79 ca đạt**: 64 ca lõi thuật toán/dữ liệu, 10 ca evaluation và 5 ca ứng dụng. Phần lõi kiểm tra ma trận Euclid; route hợp lệ/không hợp lệ; kho ở hàng bất kỳ; OX và mutation bảo toàn hoán vị; elitism; tái lập seed; NN; ngân sách Random Search; CSV; tham số không hợp lệ; JSON; dừng sớm; một khách hàng. Exact được đối chiếu độc lập với liệt kê hoán vị trên mẫu nhỏ. Hình vuông cạnh 1 có nghiệm chính xác 4. Các kiểm thử không giả định GA luôn tìm được optimum. Bản bổ sung mô phỏng đã đạt 117 ca Python và 24 ca JavaScript, trình bày ở phần 12.1.

AppTest đã kiểm tra cả sáu trang, việc chuyển snapshot không tăng số lần chạy và giữ nguyên kết quả, nhập tọa độ thủ công rồi giải hình vuông bằng 4, dữ liệu sai không thay dataset hợp lệ, benchmark từ giao diện và dịch vụ xuất PNG/JSON. Máy chủ Streamlit đã khởi chạy tại `127.0.0.1:8501`; endpoint health trả HTTP 200. `pip check` xác nhận dependencies không có xung đột được phát hiện. Đây là kiểm tra thực thi và cấu trúc giao diện; **chưa kiểm tra trực quan trong trình duyệt** vì môi trường công cụ không cung cấp browser khả dụng. AppTest không thay thế kiểm tra bố cục, tương phản hoặc thao tác thực trên nhiều kích thước màn hình.

12.1. Thiết kế và triển khai mô phỏng hoạt động phương tiện giao hàng

Mục đích và luồng sử dụng

Trang Delivery Simulation bổ sung góc nhìn vận hành cho một tuyến đã tính. Từ Dashboard, người dùng mở mô phỏng, chọn kết quả của một bộ giải đã chạy rồi nhấn Play. Xe van SVG xuất phát tại kho, đi qua từng khách hàng đúng thứ tự, dừng giao hàng và quay về kho. Thay kết quả mô phỏng không tính lại GA, NN hay Random Search. Giao đủ đơn chưa được coi là hoàn tất nếu xe chưa quay về kho.

Lịch trình và nội suy

Python kiểm tra tuyến khép kín và đối chiếu khoảng cách với dataset, sau đó tạo một lịch trình gồm các pha di chuyển và dừng. Với chiều dài đoạn `L_i`, vận tốc cơ sở `v > 0`, thời gian dừng mỗi khách `d >= 0`, thời lượng là `T = sum(L_i / v) + n*d`. Trong đoạn đang đi, tỷ lệ u xác định vị trí `P = (1-u)*P_start + u*P_end`; hướng lấy từ `atan2(dy, dx)`. Đoạn dài bằng 0 không được chia cho 0; lịch trình vẫn xử lý giao hàng và thứ tự sự kiện.

Thời điểm mô phỏng là nguồn chung cho vị trí, quãng đường, pha và số đơn đã giao. Trong thời gian giao, đồng hồ tăng nhưng khoảng cách không đổi. Thanh tua đo phần thời gian đã qua, còn Distance covered đo phần quãng đường; hai tỷ lệ này có thể khác nhau. Thời gian mô phỏng và vận tốc lựa chọn không được diễn giải thành thời gian giao hàng ngoài thực tế.

Điều khiển, đồng bộ và phục hồi

Trạng thái phát gồm idle, playing, paused, stopped, completed; pha phương tiện gồm moving, delivering, at_depot. Pause và Stop giữ nguyên vị trí; Restart đưa về thời điểm 0. Tua tiến hoặc lùi đều tạm dừng và suy ra lại toàn bộ trạng thái. Hệ số 0.5x-4x tăng tốc cả di chuyển và dừng mà không đổi tuyến. Vận tốc cơ sở và thời gian dừng được giữ chung giữa các thuật toán để so sánh có ý nghĩa.

Frontend JavaScript/SVG chạy trong component Streamlit cục bộ; không dùng vòng lặp Python để cập nhật frame. Xe, bảng trạng thái, màu tuyến và timeline đọc cùng snapshot. Checkpoint được gửi về Python ở các thao tác điều khiển; browser sessionStorage lưu con trỏ khoảng một lần mỗi giây. Khi remount, checkpoint hợp lệ khôi phục ở paused; đổi dataset, kết quả, lần chạy hoặc cấu hình lịch trình sẽ reset. Sự kiện có ID ổn định, được lọc theo timestamp nên tua lại không nhân đôi đơn giao. Ẩn tab tạm dừng đồng hồ.

Kiểm thử và giới hạn xác minh

Bản bổ sung đã chạy thành công 117 kiểm thử Python và 24 kiểm thử JavaScript. Trong đó, 33 kiểm thử Python mới xác minh lịch trình/nội suy và đối chiếu với JavaScript; ba kiểm thử ứng dụng xác minh điều hướng, chuyển kết quả, cấu hình và checkpoint. Hai ca kiểm thử còn xác minh ký hiệu nhân trong công thức khi xuất PDF. Các kiểm thử JavaScript gồm fake clock và thao tác các handler thật trong DOM: Play/Pause/Resume, Stop, Restart, tua, tốc độ, sự kiện, chọn địa điểm, SVG và giao tiếp component.

Fixture tính tay có tam giác cạnh 3, 4, 5, kho nằm ở hàng 1; tuyến dài 12, vận tốc 1 và hai lần giao dài 2 giây nên tổng thời lượng là 16. Tại giây 7, xe ở `(3,2)`, đã đi 5 đơn vị và giao xong một khách. Fixture này kiểm tra cả ranh giới pha giao hàng và đoạn quay về kho. Có thêm trường hợp toàn bộ hành trình

thời lượng 0, tham số sai, bảo toàn tổng khoảng cách và quay lại sau tua lùi.

Kiểm tra máy chủ thật đã mở trang mô phỏng qua giao thức Streamlit, nhận lịch trình GA 20 khách hàng dài 372,610897 đơn vị với thời lượng khoảng 30 giây, và xác nhận HTML, hai module JavaScript cùng CSS đều trả HTTP 200. Đây là một lần kiểm tra chức năng riêng, không thay thế hoặc bổ sung số liệu vào benchmark thuật toán ở phần 14.

Môi trường công cụ không có trình duyệt để chụp và xem animation thực tế, vì vậy chưa khẳng định chất lượng responsive hoặc FPS thực đo. Hình screenshot/GIF chưa được xuất. Giới hạn mô hình vẫn là một xe, một kho và khoảng cách Euclid; không có mạng đường, tải trọng hay giao thông. Thiết kế đầy đủ nằm trong `docs/simulation_design.md`; trạng thái xác minh tại `docs/simulation_verification.md`.

13. Thiết lập thực nghiệm

Benchmark dùng 5, 10, 20, 30 khách hàng, mỗi kích thước có một dataset cố định. Khách hàng được lấy đều trên [0,100)², kho tại `(50,50)`. Seed dataset là 1000 + số_khách_hàng. GA và Random Search chạy 10 lần trên mỗi dataset; NN, NN + 2-opt và Held-Karp (nếu không vượt 12 khách hàng) chạy một lần vì xác định.

Cấu hình mặc định: population 100; generations 200; crossover 0,9; mutation 0,2; elitism 2; tournament 3; patience 0; base seed 42. Mỗi lần GA sử dụng 100 + 200×98 = 19.700 đánh giá. Seed GA/Random Search được sinh độc lập bằng SeedSequence([42, số_khách_hàng, lần_lặp]).generate_state(2) và ghi trong metadata. Random Search được cấp đúng ngân sách của GA tương ứng, không dùng một ngân sách tùy ý thuận lợi cho phương pháp nào.

Std distance dùng độ lệch chuẩn mẫu (`ddof=1`) cho các lần ngẫu nhiên. Phương pháp xác định có một lần đo được ghi std 0 theo quy ước phần mềm, không phải bằng chứng về biến thiên runtime. Thời gian đo bằng `perf_counter` trong từng bộ giải; so sánh ngân sách không đồng nghĩa chi phí CPU bằng nhau. Metadata lưu phiên bản Python, thư viện và nền tảng máy.

14. Kết quả thực nghiệm

Chiến dịch thực nghiệm đã thực thi và lưu đủ **40 lần GA, 40 lần Random Search**, cùng 4 NN, 4 NN + 2-opt và 2 Held-Karp: tổng cộng **90 lần chạy thật**. Metadata được tạo lúc 10:31 ngày 07/11/2026 (UTC+7). Môi trường: Windows 11, Python 3.14.4, NumPy 2.5.4, Pandas 3.1.1, Plotly 6.9.0, Streamlit 1.67.0 và Matplotlib 3.11.2. Các bảng dưới làm tròn từ CSV gốc, không thay số liệu gốc.

Chất lượng tuyến của hai thuật toán ngẫu nhiên (mỗi hàng 10 lần; đơn vị tọa độ):

Khách hàng	Thuật toán	Best	Mean	Worst	SD mẫu
5	GA	255,463	255,463	255,463	0,000
5	Random Search	255,463	255,463	255,463	0,000
10	GA	368,393	368,833	372,799	1,393
10	Random Search	390,827	404,100	413,779	8,585
20	GA	371,117	381,025	407,481	11,322
20	Random Search	625,786	659,655	683,923	20,331

Khách hàng	Thuật toán	Best	Mean	Worst	SD mẫu
30	GA	477,395	493,379	519,290	13,144
30	Random Search	1075,737	1131,349	1166,266	30,771

Các mốc xác định (một lần chạy mỗi phương pháp/dataset):

Khách hàng	NN	NN + 2-opt	Optimum Held-Karp
5	255,498	255,463	255,463
10	422,285	387,709	368,393
20	380,683	374,438	Không tính
30	609,247	542,152	Không tính

Runtime trung bình và ngân sách: mỗi lần GA và Random Search đều có đúng **19.700 đánh giá tuyến**. Các phương pháp xác định có một lần đo, nên runtime của chúng trong bảng là giá trị đo đơn lẻ, không phải trung bình của nhiều lần.

Khách hàng	GA (giây)	Random (giây)	NN (giây)	2-opt (giây)	Exact (giây)
5	0,554662	0,049896	0,000183	0,000213	0,000229
10	0,694632	0,063480	0,000141	0,001769	0,019021
20	1,011534	0,088152	0,000194	0,023280	Không tính
30	0,863258	0,074160	0,000209	0,204081	Không tính

NN báo 1 lần đánh giá tuyến hoàn chỉnh mỗi dataset; 2-opt lần lượt báo 12, 111, 409, 2485 đánh giá tuyến ứng viên. Held-Karp báo 170 và 23.060 chuyển trạng thái DP. Các đơn vị này không được gộp thành một xếp hạng hiệu quả đánh giá chung.

Cải thiện và khoảng cách tới tối ưu (trung bình 10 lần):

Khách hàng	GA so với NN (%)	Random so với NN (%)	GA gap (%)	Random gap (%)
5	+0,013	+0,013	0,000	0,000
10	+12,658	+4,306	0,120	9,693
20	-0,090	-73,282	Không tính	Không tính
30	+19,018	-85,696	Không tính	Không tính

Toàn bộ bảng lấy từ `results/final/benchmark_summary.csv`. Dữ liệu từng lần, seed, cấu hình, tuyến và history nằm trong `results/raw/benchmark.json` và `results/raw/benchmark_runs.csv`. Hình tĩnh gồm hai biểu đồ tổng hợp và bốn hình tuyến/hội tụ. HTML tương tác dùng chung `plotly.min.js`; giữ nguyên thư mục figures khi di chuyển để xem ngoại tuyến.

15. Thảo luận

Ở **5 khách hàng**, GA và Random Search đều đạt optimum trong cả 10 lần. Không gian chỉ có 120 hoán vị khách hàng trong khi ngân sách là 19.700, nên kết quả này không phân biệt khả năng tìm kiếm ở quy mô lớn. Đây là nhận xét từ cấu trúc bài toán, không phải bảo đảm mọi lần ngẫu nhiên sẽ thăm hết không gian. NN kém optimum rất ít; NN + 2-opt cũng đạt optimum trong lần đo.

Ở **10 khách hàng**, GA đạt optimum 9/10 lần; một lần có khoảng cách 372,799 làm mean gap tăng lên 0,120%. Random Search có mean gap 9,693%; GA có tuyến ngắn hơn ở cả 10 cặp so sánh. NN + 2-opt đạt 387,709, tốt hơn NN nhưng vẫn cao hơn optimum 5,243%. Điều này thể hiện giới hạn của tìm kiếm cục bộ từ tuyến NN trên chính instance này.

Ở **20 khách hàng**, kết quả minh họa rõ rằng GA không luôn thắng NN: GA thua NN **3/10 lần**, mean distance 381,025 cao hơn NN 380,683, tương ứng improvement âm -0,090%. Tuyến GA tốt nhất 371,117 lại ngắn hơn NN + 2-opt 374,438. Vì vậy, chỉ báo cáo best sẽ che đi độ biến thiên và làm kết luận khác với dùng mean. Random Search kém rõ trên ngân sách đang xét; GA thắng cả 10 cặp, nhưng không có optimum để xác định gap chính xác.

Ở **30 khách hàng**, GA có mean distance 493,379, cải thiện 19,018% so với NN và ngắn hơn kết quả NN + 2-opt 542,152. GA thắng Random Search ở cả 10 cặp. Kết quả này chỉ hỗ trợ ưu thế chất lượng tuyến trên instance và cấu hình đã đo, không chứng minh ưu thế trên mọi bài toán 30 khách hàng.

Tổng cộng GA có **30 cặp tốt hơn và 10 cặp ngang bằng Random Search**, không có cặp thua theo ngưỡng sai số so sánh. So với NN, GA thua 3/40 lần, đều thuộc instance 20 khách hàng. Kết quả quan sát phù hợp với việc tìm kiếm quần thể có thể sử dụng ngân sách hiệu quả hơn lấy mẫu độc lập ở các instance lớn hơn, nhưng không tách riêng đóng góp từng toán tử.

Runtime GA cao hơn Random Search trong cả bốn kích thước, dù số đánh giá bằng nhau. Chọn lọc và toán tử tiến hóa có overhead, còn Random Search được đánh giá theo lô; điều này giải thích vì sao cân bằng evaluations không cân bằng thời gian. Số đo GA ở 30 khách hàng thấp hơn ở 20 khách hàng không được diễn giải thành độ phức tạp giảm: runtime chịu ảnh hưởng môi trường và cách thực thi, cần nhiều phép đo kiểm soát hơn để kết luận về tốc độ theo quy mô.

Improvement so với NN có dấu: 100×(NN-distance)/NN. Gap tới optimum là 100×(distance-exact)/exact, chỉ có ở 5 và 10 khách hàng. Các trường hợp mẫu số 0 được đánh dấu riêng; không gán một tỷ lệ hữu hạn tùy ý.

Không suy ra một thuật toán luôn tốt hơn từ một instance mỗi kích thước. Các toán tử GA có chi phí quản lý quần thể, selection, crossover và mutation; NN khai thác hình học trực tiếp; Random Search có thể được triển khai vector hóa hiệu quả. Vì vậy, chất lượng tuyến và runtime là các khía cạnh cần báo cáo riêng. Thua baseline có thể phản ánh ngân sách hữu hạn hoặc tính ngẫu nhiên, nhưng thí nghiệm này không xác định duy nhất cơ chế gây ra từng thất bại.

16. Hạn chế

Chỉ có một hình học ngẫu nhiên mỗi kích thước. Mười seed đo biến thiên tìm kiếm trên cùng hình học, không phải mười bài toán giao hàng độc lập. Bộ tham số không được tối ưu riêng cho từng kích thước. Không thực hiện kiểm định ý nghĩa thống kê hoặc suy rộng cho mọi phân bố tọa độ.

Runtime phụ thuộc máy, tải hệ thống, overhead Python và thư viện; các phương pháp xác định mới có một phép đo runtime. Exact chỉ cung cấp optimum ở dữ liệu nhỏ. Bài toán thực tế còn có mạng đường, tải trọng, thời gian và nhiều xe mà mô hình hiện tại không thể hiện. Dữ liệu giao diện được giữ trong session; người dùng cần tải tệp để lưu lâu dài.

17. Hướng phát triển

Cần mở rộng benchmark trên nhiều dataset độc lập cho mỗi kích thước và nhiều phân bố hình học, ghi cả thời gian CPU, kiểm soát môi trường và thiết kế kiểm định thích hợp. Có thể nghiên cứu tác động của population, mutation, tournament và early stopping theo một kế hoạch ablation định trước.

Về thuật toán, có thể bổ sung GA + 2-opt với nhãn riêng, đa dạng hóa quần thể và so sánh ngân sách công bằng. Về mô hình, mở rộng sang dữ liệu đường bộ hoặc VRP đòi hỏi thay hàm khoảng cách và ràng buộc, không chỉ thay giao diện. Về sản phẩm, có thể bổ sung tác vụ nền, lưu trữ bền vững và truy cập nhiều người dùng khi cần.

18. Kết luận

Dự án hiện thực một quy trình TSP có thể nhập dữ liệu, chạy GA, kiểm tra tính hợp lệ, phát lại lịch sử, so sánh đối chứng và lưu thí nghiệm tái lập. Tính đúng của lời giải được tách khỏi chất lượng tối ưu: một route hợp lệ không tự chứng minh là route ngắn nhất. Exact solver là mốc kiểm chứng ở quy mô nhỏ; GA và 2-opt được đánh giá như heuristic.

Giá trị của dự án nằm ở sự kết hợp giữa mô hình rõ ràng, thuật toán tự triển khai, giao diện dùng được, kiểm thử tự động và dữ liệu đo có thể kiểm tra. Kết luận về ưu thế thuật toán cần giới hạn trong dữ liệu và ngân sách đã ghi nhận.

19. Tài liệu tham khảo

[1] Holland, J. H. (1992). *Adaptation in Natural and Artificial Systems: An Introductory Analysis with Applications to Biology, Control, and Artificial Intelligence*. MIT Press. DOI: 10.7551/mitpress/1090.001.0001.

[2] Held, M., & Karp, R. M. (1962). *A Dynamic Programming Approach to Sequencing Problems*. Journal of the Society for Industrial and Applied Mathematics, 10(1), 196-210. DOI: 10.1137/0110015.

[3] Streamlit. *The AppTest class*. Tài liệu chính thức: docs.streamlit.io/develop/api-reference/app-testing/st.testing.v1.apptest. Truy cập ngày 07/11/2026.

Hai nguồn học thuật hỗ trợ nền tảng GA và quy hoạch động. Cấu hình, quy tắc đếm evaluations và kết quả trong báo cáo là thiết kế/đo lường của dự án; không quy chúng cho tài liệu tham khảo.

20. Phụ lục

Khởi chạy: `python -m streamlit run app.py` sau khi cài `requirements.txt` trong môi trường ảo.

Kiểm thử: `python -m pytest`. **Thực nghiệm:** `python scripts/run_experiments.py --repeats 10 --customers 5 10 20 30 --population 100 --generations 200 --patience 0 --seed 42 --write-samples`.

Schema CSV: `id,name,x,y,is_depot`. **Schema kết quả:** algorithm, route, distance, runtime_seconds, evaluations, seed, parameters, history. **Đơn vị:** tọa độ Euclid và giây; route dùng chỉ số hàng dataset, có kho ở hai đầu.

Tệp kiểm tra: `docs/architecture.md`, `docs/algorithm_design.md`, `docs/experimental_protocol.md`, `docs/user_guide.md`; `results/final/metadata.json` ghi cấu hình và môi trường; `results/final/findings.md` tạo nhận xét từ dữ liệu đo; `results/figures/` có PNG và HTML.

Định dạng tài liệu: nội dung gốc đầy đủ nằm trong Markdown; báo cáo PDF được xuất từ nội dung này, còn thuyết trình có nội dung slide Markdown. Chưa xuất Word (`.docx`) và PowerPoint (`.pptx`) vì môi trường artifact hiện tại không hỗ trợ quy trình xuất các định dạng đó. GA Lab dùng thanh trượt xem lại thế hệ; Delivery Simulation có animation phương tiện chạy theo tuyến đã tính.

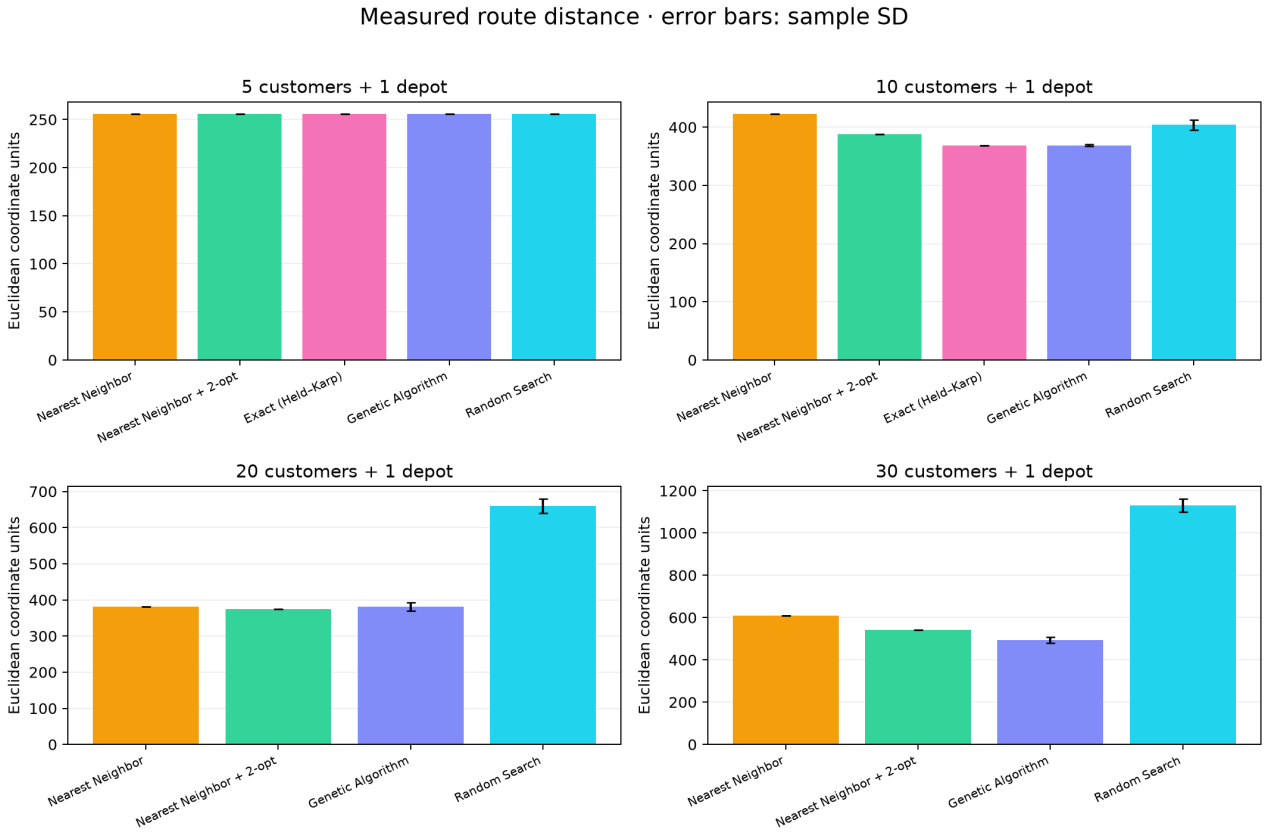
Quy trình tái lập: giữ đúng dataset đã xuất, seed từng thuật toán, cấu hình và phiên bản thư viện; chạy lại rồi so route/distance/history. Không yêu cầu runtime trùng từng chữ số. Không thay kết quả thiếu bằng giá trị giả và không gọi heuristic tốt nhất là optimum khi chưa giải exact.

Measured benchmark appendix

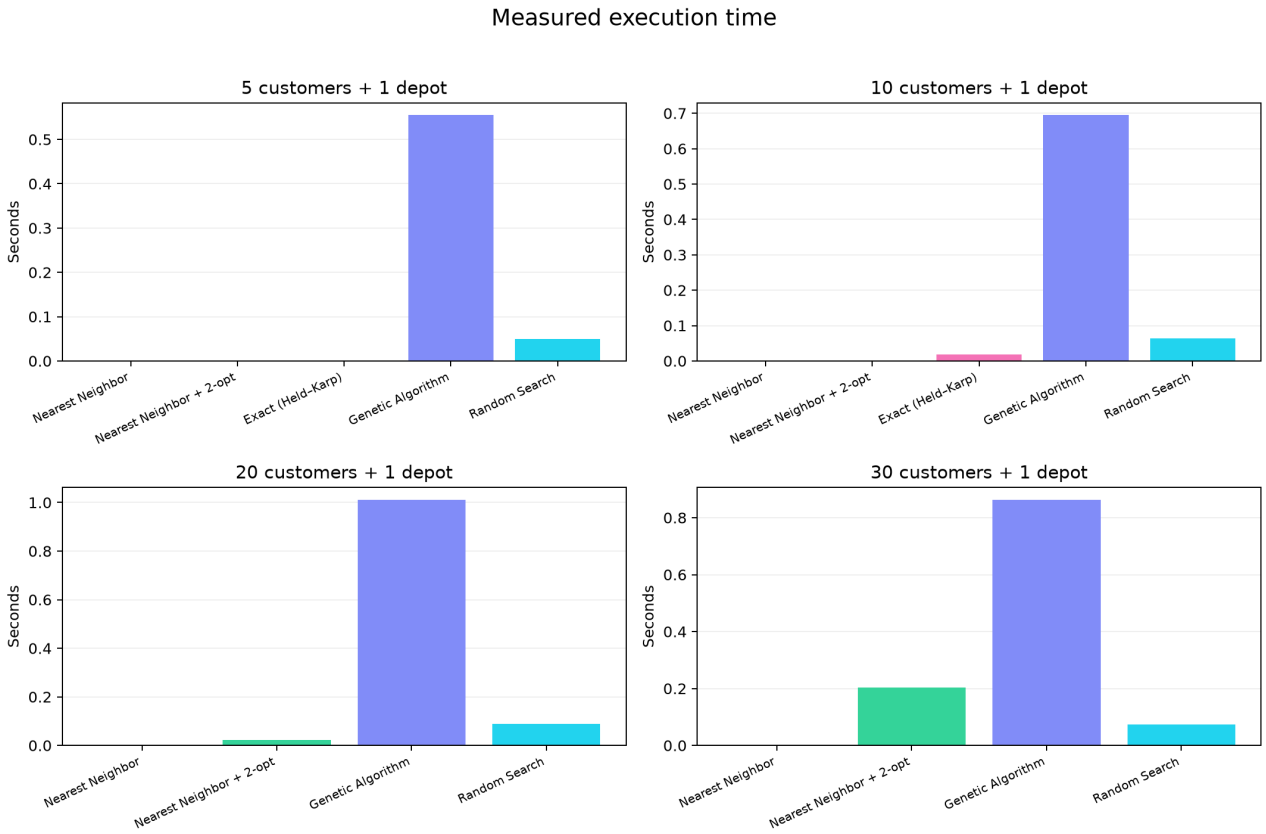
Distances use abstract Euclidean units. Runtime is measured wall time in seconds. Standard deviation uses repeated runs. The complete machine-readable table and raw seeds accompany this report.

n	Algorithm	Runs	Mean	Std	Best	Time s
5	NN	1	255.50	0.00	255.50	0.0002
5	NN + 2-opt	1	255.46	0.00	255.46	0.0002
5	Exact	1	255.46	0.00	255.46	0.0002
5	GA	10	255.46	0.00	255.46	0.5547
5	Random	10	255.46	0.00	255.46	0.0499
10	NN	1	422.28	0.00	422.28	0.0001
10	NN + 2-opt	1	387.71	0.00	387.71	0.0018
10	Exact	1	368.39	0.00	368.39	0.0190
10	GA	10	368.83	1.39	368.39	0.6946
10	Random	10	404.10	8.59	390.83	0.0635
20	NN	1	380.68	0.00	380.68	0.0002
20	NN + 2-opt	1	374.44	0.00	374.44	0.0233
20	GA	10	381.02	11.32	371.12	1.0115
20	Random	10	659.66	20.33	625.79	0.0882
30	NN	1	609.25	0.00	609.25	0.0002
30	NN + 2-opt	1	542.15	0.00	542.15	0.2041
30	GA	10	493.38	13.14	477.40	0.8633
30	Random	10	1131.35	30.77	1075.74	0.0742

Source: results/final/benchmark_summary.csv. Exact-solver evaluation counts describe DP transitions and should not be compared to counts of full-tour evaluations.



Mean route distance across repeated runs



Mean runtime across repeated runs